A Inteligência é Artificial mas a mudança é Humana

Eliezer Carvalho †
Infraestruturas de Portugal
eliezer.carvalho@infraestruturasdeportugal.pt

António Simões
Infraestruturas de Portugal
antonio.ssimoes@infraestruturasdeportugal.pt





Abstrato

A Inteligência Artificial está a adquirir uma presença cada vez mais notória na sociedade e nas organizações. A rápida evolução tecnológica e o advento de modelos cada vez mais capazes ampliaram significativamente as possibilidades de aplicação desta tecnologia, no entanto, a transição da investigação para a adoção efetiva pelas empresas continua complexa, condicionada por questões tecnológicas e requisitos de segurança, privacidade, infraestrutura e dados.
Propomos, neste trabalho, explorar de que forma é possível reduzir a distância entre o desenvolvimento de sistemas de Inteligência Artificial e a sua adoção efetiva pelas organizações, partindo da experiência e do trabalho desenvolvido ao longo de um estágio curricular de quatro meses. Neste período, foram exploradas e desenvolvidas soluções de Inteligência Artificial. Procedeu-se ao estudo aprofundado de técnicas de inferência, quantização, motores de inferência, engenharia de contexto e geração de dados sintéticos. Este estudo culminou no desenvolvimento de duas soluções, designadas HOW TO RAG e ASSOBIO.
Ambos os sistemas foram avaliados através de benchmarks e métricas específicas, com o objetivo de compreender as suas capacidades, limitações e aplicabilidade em contexto empresarial. Mais do que questionar aquilo que a Inteligência Artificial é capaz de fazer, importa questionar onde deve ser utilizada e porquê. Procurámos explorar precisamente esta questão, partindo da dificuldade que as organizações enfrentam em transformar uma evolução tecnológica extremamente rápida em aplicações que gerem valor real. A existência de modelos mais capazes não implica, por si só, melhores soluções. O valor surge quando existe um problema concreto, uma necessidade clara e uma utilização tecnicamente e economicamente justificável.
Com um grande poder também surge uma grande responsabilidade.

† Trabalho realizado no âmbito de um estágio curricular na empresa Infraestruturas de Portugal, na Direção de Acessibilidade, Telemática e ITS. O estágio decorreu entre Maio e Outubro de 2026, contando com a orientação de António Simões.

1 Introdução

A Inteligência Artificial é, sem dúvida, uma tecnologia com um tremendo potencial transformador. Esta tecnologia, à semelhança da Computação, da Eletricidade ou das Baterias de Lítio, vai deixar a sua marca indelével na sociedade. O impacto desta transformação é evidente para as pessoas. Nas organizações, existe uma relação amor-ódio.

Em Junho de 2026, apenas 11,5% das empresas portuguesas adotaram Inteligência Artificial nos seus processos de negócio [1]. A mensagem é clara: as organizações continuam a enfrentar uma dificuldade real em transformar a evolução tecnológica em adoção efetiva.
A realidade na Infraestruturas de Portugal não é alheia a este desafio.

A adoção da Inteligência Artificial por uma organização não é determinada pela utilização de ChatGPT ou Copilot pelos seus colaboradores. Isto é acesso à tecnologia, não integração. A verdadeira adoção exige a capacidade de construir soluções próprias a partir dos modelos disponíveis e direcioná-las para problemas concretos da organização. Isso implica infraestrutura, dados, capacidade computacional e, acima de tudo, pessoas capazes de desenvolver, adaptar e integrar estes sistemas.
Neste momento, em que a tecnologia está cada vez mais acessível, esta necessidade é particularmente relevante. Os custos diminuem, as capacidades aumentam e a utilização dos modelos cresce. A Figura 1 mostra claramente essa redução do custo ao longo do tempo.

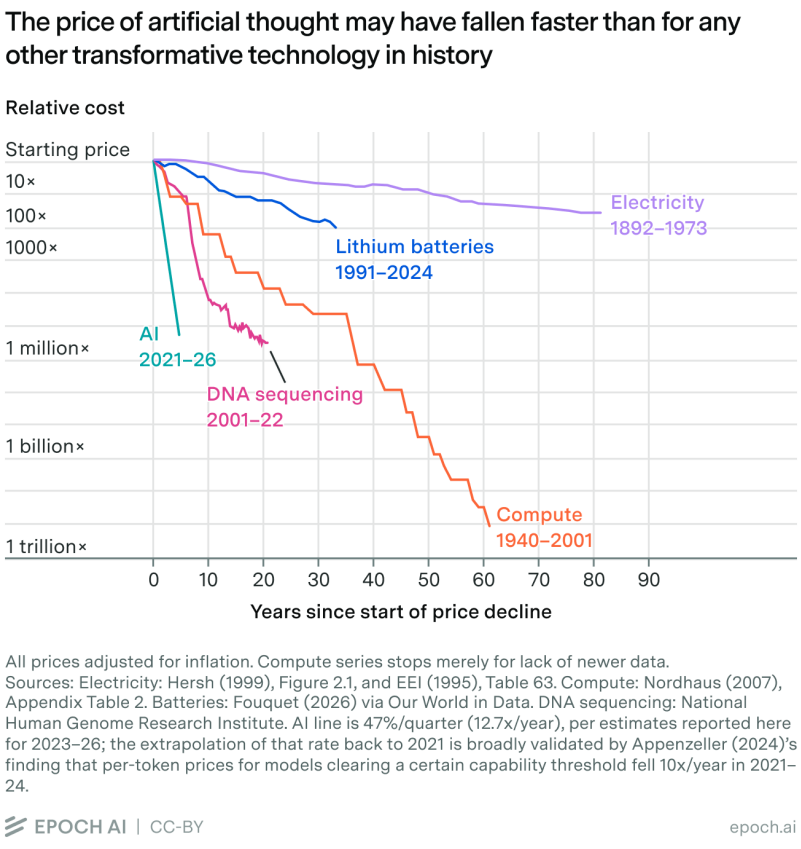


Figura 1: O custo para atingir um determinado nível de desempenho com Inteligência Artificial diminui cerca de 47% por trimestre desde 2023, ou seja, 13 vezes por ano.
Esta queda de preços é quatro vezes mais rápida do que DNA Sequencing, seis vezes mais rápida do que a Computação, 18 vezes mais rápida do que as Baterias de Lítio e 54 vezes mais rápida do que a Eletricidade [2].

A rápida evolução desta tecnologia cria uma oportunidade que as organizações já não podem adiar. Os modelos são cada vez mais capazes e o seu custo de utilização continua a diminuir. O acesso deixou de ser o principal obstáculo, a capacidade para os transformar em soluções úteis é agora o verdadeiro desafio. É precisamente essa capacidade que desenvolvemos ao longo deste trabalho.

Durante quatro meses, estudámos, experimentámos e aplicámos com sucesso diferentes técnicas e tecnologias necessárias à construção de sistemas baseados em modelos de Inteligência Artificial. O trabalho não se ficou pela exploração tecnológica, mas culminou no desenvolvimento de duas soluções, HOW TO RAG e ASSOBIO, orientadas para problemas concretos da organização.

2 Fase de Investigação

P1: Como podemos hospedar modelos de Inteligência Artificial localmente de maneira soberana ?

Numa fase crisálida do estágio, a investigação partiu de uma necessidade concreta: perceber se era possível executar modelos de linguagem localmente e sob controlo da organização. A soberania sobre os modelos, os dados e a infraestrutura é, desde o início, uma questão central.

A resposta não pode limitar-se a um simples "sim". Hospedar um modelo localmente só é relevante se a execução for tecnicamente viável e apresentar um desempenho compatível com a utilização pretendida. Esta fase da investigação resultou na documentação *"A Three-Dimensional Analysis for LLM Deployment"* [3], onde sistematizámos esta ideia através de uma abordagem tridimensional baseada em três dimensões: Performance, Tempo e Memória.

P2: Temos os modelos do nosso lado. Como os orientamos para resolver os problemas que nos interessam ?

Os modelos não conhecem, à partida, as particularidades de cada organização, os seus processos ou a informação necessária para resolver um problema específico. O seu conhecimento interior não garante resultados úteis. Para os orientar para uma tarefa concreta, é necessário fornecer-lhes informação exterior.

O problema não está apenas no modelo. Está também na forma como o problema lhe é apresentado. Que informação fornecemos ? O que selecionamos ? Como a organizamos ? E de que forma garantimos que o modelo dispõe dos elementos necessários para responder ? *"Um prompt maior não é obrigatoriamente um prompt melhor. O objetivo do Context Engineering é maximizar a qualidade da informação fornecida e não a sua quantidade. Um contexto curto, relevante e bem organizado reduz custos, melhora a precisão e aproveita melhor a context window do modelo"* [4].

P3: Se os dados são o novo petróleo, quem os gera controla a refinaria ?

Os modelos não surgem do vazio, apesar das narrativas comerciais das grandes empresas tecnológicas tentarem fazer-nos acreditar no contrário. O seu desenvolvimento assenta em enormes quantidades de dados, como demonstram os datasets RefinedWeb, C4 e RedPajama. A escala destes recursos é impressionante, mas levanta uma questão diferente para as organizações: E quando os dados de que precisamos não existem ?

Treinar um modelo, avaliar um sistema ou testar o seu comportamento em cenários específicos exige dados adequados à tarefa.

Recolher mais dados nem sempre é possível e reutilizar os existentes nem sempre é suficiente. A capacidade de gerar dados sintéticos é a solução para esta limitação, permitindo criar exemplos orientados para necessidades concretas.

Esta capacidade foi explorada durante a fase de investigação. Estudámos a geração de dados sintéticos para diferentes aplicações, dando particular atenção à avaliação de sistemas, uma das utilizações mais recorrentes ao longo do estágio [5].

3 How To RAG

A investigação estabeleceu as bases técnicas. Seguiu-se a aplicação a um problema concreto, através do desenvolvimento do sistema How To RAG, o primeiro dos dois sistemas construídos ao longo do estágio.

3.1 Motivações e Conceitos

A informação é um dos ativos mais importantes de uma organização, mas nem toda pode ser disponibilizada a serviços externos. Esta limitação é especialmente relevante em infraestruturas críticas, onde a confidencialidade dos dados e o controlo sobre a sua utilização são requisitos fundamentais. Os documentos internos são uma fonte de conhecimento que os modelos de linguagem podem explorar. Permitir que um modelo responda a perguntas com base nessa informação abre novas possibilidades de acesso ao conhecimento organizacional.

Retrieval-Augmented Generation (RAG) é uma arquitetura que integra recuperação de informação *(retrieval)* e geração de texto *(generation)* num único processo. O objetivo, é fornecer aos modelos de linguagem informação externa relevante para a tarefa em execução, permitindo fundamentar as respostas em fontes de conhecimento fidedignas.

Vamos imaginar que temos um conjunto de documentos e queremos fazer perguntas sobre o seu conteúdo a um modelo de linguagem. A solução mais fácil é fornecer-lhe todos os documentos e pedir-lhe que responda às nossas perguntas com base na informação disponibilizada. No entanto, é importante ter em conta que, à medida que o número e a dimensão dos documentos aumentam, esta abordagem torna-se cada vez menos viável. Os modelos têm uma janela de contexto limitada. Isto restringe a quantidade de informação que conseguem processar de uma só vez. Além disso, despejar mais informação não garante respostas melhores: o modelo pode não identificar os elementos relevantes ou gerar respostas que não são sustentadas pelos documentos fornecidos. **Tal como os humanos ao ler um livro, tendemos a recordar melhor a informação que lemos à duas páginas anteriores do que aquela que lemos nas páginas iniciais, os modelos de linguagem também podem ter dificuldade em utilizar de forma consistente toda a informação presente num contexto extenso.**

Com RAG, o sistema não fornece todos os documentos ao modelo para cada pergunta. Em vez disso, procura identificar os excertos mais relevantes para a *query* fornecida pelo utilizador. Estes excertos, denominados de *chunks*, são fornecidos ao modelo como contexto. Desta forma, o modelo não precisa de processar toda a documentação a cada pergunta. Recebe apenas a informação mais útil para responder.

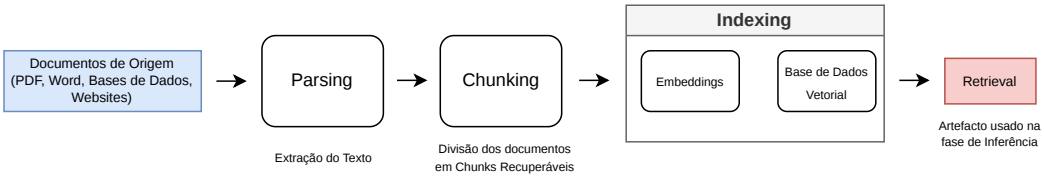


Figura 2: No ciclo de vida da Fase de Indexação, os documentos fornecidos pelo utilizador são processados através de **Parsing**, que extrai o conteúdo textual. Este é, posteriormente, dividido em *chunks* (**Chunking**). Depois, são gerados *embeddings* para cada excerto, que ficam armazenados numa base de dados vetorial. O resultado é um artefacto que permite recuperar os chunks mais relevantes consonte a *query* do utilizador.

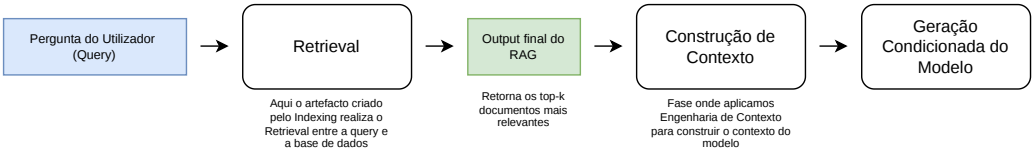


Figura 3: No ciclo de vida da Fase de Inferência, a *query* do utilizador é utilizada na fase de **Retrieval** para recuperar, normalmente através da Similaridade do Cosseno, os top-k *chunks* mais relevantes. Estes são posteriormente utilizados na **Construção do Contexto**, recorrendo a técnicas de Engenharia de Contexto.

A Similaridade do Cosseno assume aqui um papel importante e esclarecedor, sendo definida por:

$$\text{sim}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \, \|\mathbf{B}\|}$$

Onde:

- $\mathbf{A}$ e $\mathbf{B}$ representam os *embeddings* (vetores).
- $\mathbf{A} \cdot \mathbf{B}$ representa o produto escalar entre os vetores.
- $\|\mathbf{A}\| \, \|\mathbf{B}\|$ representa as normas L2 (Norma Euclidiana) dos vetores.

Assim, numa condição normal, podemos representar a geração da resposta por um modelo de linguagem através de uma distribuição de probabilidades condicionada à pergunta do utilizador:

$$\boxed{p_\theta(y \mid q)}$$

Onde:

- θ representa o espaço probabilístico.
- y representa a resposta correta esperada.
- q representa a pergunta do utilizador.

Com RAG procurámos complementar a consulta com informação externa relevante, recuperada a partir de uma coleção documental definida pelo utilizador. Definimos assim o problema:

$$\boxed{p_\theta(y \mid q, D)}$$

Onde:

- D representa o corpus de documentos disponíveis.

3.2 Arquitetura

As arquiteturas apresentadas nas Figuras 2 e 3 correspondem à configuração *vanilla* de sistemas RAG. A arquitetura proposta supera as limitações convencionais, tornando o How To RAG mais robusto na recuperação de informação relevante.

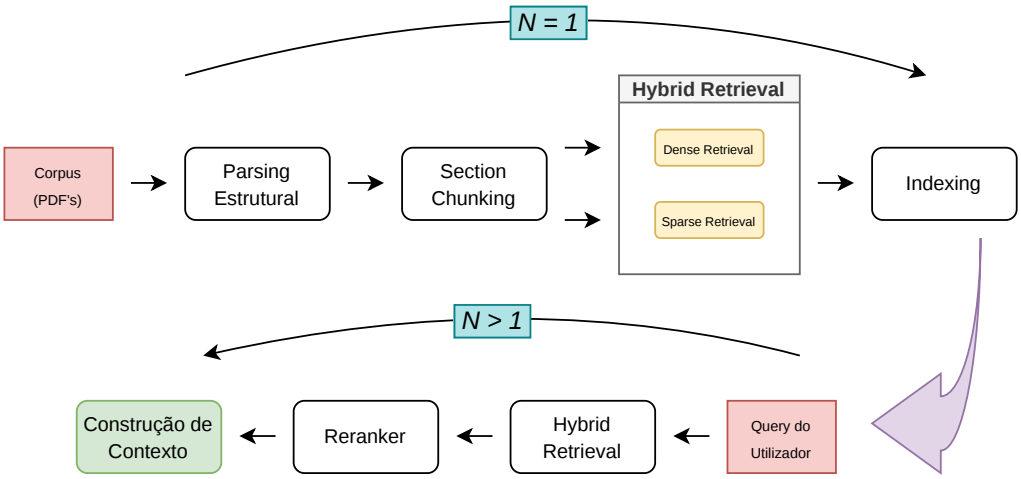


Figura 4: Arquitetura do sistema How To RAG.

O fluxo na parte superior da figura é convencional até à etapa de *Hybrid Retrieval*. Aqui, a arquitetura ganha outra dimensão, ao combinar duas abordagens distintas de recuperação: **Dense Retrieval** e **Sparse Retrieval**.

Dense Retrieval utiliza embeddings para captar relações semânticas entre a pergunta do utilizador e os chunks, permitindo recuperar informação relevante mesmo quando a formulação difere. Sparse Retrieval à boleia do algoritmo *Best Matching 25 (BM25)*, explora correspondências lexicais, identificando chunks que partilham termos com a consulta. Os dois métodos captam sinais distintos e complementares. **Enquanto o Dense Retrieval privilegia o significado, o Sparse Retrieval valoriza a correspondência entre termos**. Em vez de escolher entre ambos, o Hybrid Retrieval combina-os.

Para esta combinação, utilizamos o algoritmo *Reciprocal Rank Fusion (RRF)*, que combina as posições atribuídas aos documentos por cada método numa ordenação conjunta. A estratégia de recuperação passa assim, a resultar da combinação de perspetivas complementares.

A fórmula utilizada é definida por:

$$\boxed{\text{RRF}(d) = \sum_{i=1}^{m} \frac{1}{k + \text{rank}_i(d)}}$$

Onde:

- d representa o chunk a avaliar.
- m representa o número de rankings a combinar.
- $rank_i(d)$ representa a posição do documento d no ranking do sistema i, começando na posição 1.
- k representa a constante de suavização. Frequentemente definida como 60.

Exemplo Ilustrativo: Em testes ao Hybrid Retrieval, uma query retornou dois chunks nas seguintes posições nos respetivos Retrieval's:

Chunk	Sparse Retrieval	Dense Retrieval
A	1	5
B	2	1

Usando $k = 60$:

$$\boxed{\text{RRF}(A) = \frac{1}{60+1} + \frac{1}{60+5} \approx 0.03178}$$

$$\boxed{\text{RRF}(B) = \frac{1}{60+2} + \frac{1}{60+1} \approx 0.03252}$$

Assim, o ranking combinado fica:

1. Chunk B ≈ 0.03252
2. Chunk A ≈ 0.03178

A figura 4 assinala ainda ($N = 1$) na Fase de Indexação, indicando que esta é executada uma única vez. Isto porque, depois de selecionar os documentos desejados, o seu conteúdo é processado e armazenado na base de dados vetorial. O artefacto resultante é reutilizado nas consultas seguintes, servindo de base ao *Retrieval*.

A Fase de Inferência, que descreve o trajeto de uma query desde a sua submissão pelo utilizador até à geração da resposta, encontra-se representada na parte inferior da figura.

O sistema inicia a fase de *Retrieval* após receber a *query*, comparando-a com os chunks previamente indexados na base de dados vetorial. Num sistema RAG convencional como o da figura 3, estes resultados são integrados diretamente no contexto fornecido ao modelo de linguagem. No How To RAG, aprofundámos este processo com sucesso, introduzindo um modelo *Cross-Encoder*.

Em contraste com o método baseado em embeddings, que representa a query de cada chunk separadamente para permitir uma comparação eficiente, um Cross-Encoder recebe a query e o conteúdo do chunk em conjunto. Esta abordagem permite ao modelo analisar diretamente a relação entre ambos, considerando o significado dos termos no seu contexto e a forma como a informação do documento responde à questão colocada. Como consequência, é possível produzir uma estimativa de relevância mais contextualizada, embora com um custo computacional superior, uma vez que cada par [**query-chunk**] tem de ser processado individualmente. Esta etapa funciona como um mecanismo de **Reranking** ou seja, os candidatos inicialmente recuperados são novamente avaliados e ordenados segundo a sua relevância para a consulta. O objetivo é refinar os resultados, reduzindo a probabilidade de os excertos mais relevantes ficarem ocultos entre candidatos menos adequados.

Na figura, esta fase é identificada por $(N > 1)$, porque a inferência é executada sempre que o utilizador submete uma nova consulta. Ao contrário da indexação, que é realizada previamente e reutilizada nas consultas seguintes, a fase de inferência repete-se a cada submissão de prompt.

3.3 Avaliação

"Sem avaliação não existe Engenharia" [6].

Como não existem sistemas perfeitos passámos à fase de avaliar o sistema construído.

Construir um sistema RAG é apenas parte do problema. A outra parte igualmente importante é determinar se o sistema recupera a informação certa e se utiliza essa informação para produzir respostas adequadas. Sem uma metodologia de avaliação, qualquer conclusão sobre a qualidade do sistema é baseada em exemplos isolados e impressões subjetivas.

Esta fase do estágio foi uma fase onde quisemos realmente aprofundar porque percebemos desde o inicio a importância de realizar uma avaliação exaustiva e completa a estes sistemas. Após várias semanas de investigação, identificámos os **Benchmarks** e os **Golden Datasets** como componentes fundamentais deste processo. Um Golden Dataset reúne exemplos de avaliação acompanhados de respostas de referência ou de outras anotações. Estas permitem determinar se os resultados produzidos pelo sistema são adequados. Um Benchmark define o procedimento de avaliação do sistema, estabelecendo os dados de teste, as métricas e os critérios de comparação.

Antes de avaliar o sistema, tivemos, portanto, de construir os dados sobre os quais essa avaliação iria assentar. Começámos por selecionar um corpus documental e recorremos a técnicas de geração de dados sintéticos para produzir dois datasets de avaliação, utilizando dois modelos de linguagem distintos.

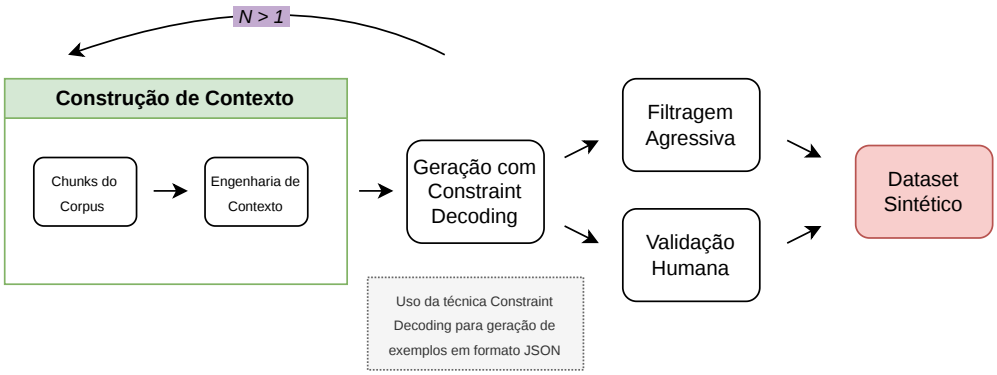


Figura 5: Pipeline usada para a construção do Golden Dataset Sintético usando dois modelos, Microsoft Phi-4 15B Q4 [7] e Mistral 7B Q4 [8].

Definido o Golden Dataset, estruturámos a avaliação do How To RAG em duas etapas distintas, correspondentes aos principais componentes do sistema: a recuperação de informação **(Retrieval)** e a geração da resposta pelo modelo de linguagem **(Generation)**.

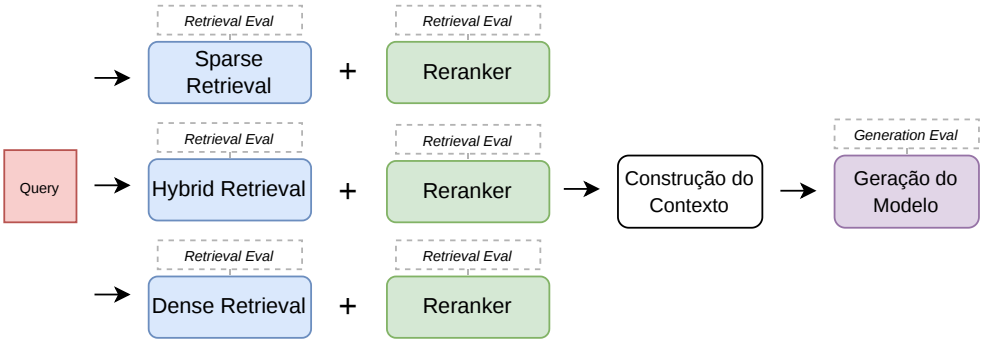


Figura 6: Comparámos as três estratégias de recuperação com o objetivo de perceber qual permitia identificar os chunks mais relevantes para cada consulta.

De seguida, avaliámos novamente estas três estratégias após a introdução do **Reranker**, procurando determinar se a reordenação dos resultados através de um modelo *Cross-Encoder* permitia melhorar a qualidade da recuperação.

A parte final do fluxo, procura avaliar a qualidade das respostas geradas pelo sistema. Para este efeito, selecionámos dois modelos de linguagem - Qwen2.5 0.5B [9] e Phi-3.5 4B Q4 [10] - incluindo deliberadamente um modelo de menor dimensão, de modo a avaliar a sua capacidade de produzir respostas adequadas com base no contexto recuperado.

É importante salientar que para a *Generation Eval*, recorremos à técnica **LLM-as-a-Judge**. Esta técnica consiste num modelo de linguagem utilizado como Evaluator que analisa as respostas produzidas pelo sistema segundo critérios de avaliação definidos no contexto. Neste processo, um modelo de maior dimensão atua como avaliador, examinando a resposta gerada à luz da query e do corpus fornecido. Foi utilizado o modelo GPT 5 como Evaluator.

A seguinte tabela reúne as métricas utilizadas e os respetivos objetivos:

Benchmark	Objetivo
Recall@k	Percentagem de todos os resultados relevantes que foram encontrados nos primeiros k resultados.
Precision@k	Percentagem dos primeiros k resultados que são relevantes.
HitRate@k	Percentagem de perguntas para as quais foi encontrado pelo menos um resultado relevante.
MRR@k	Mede quão cedo aparece o primeiro resultado relevante.
Answer Relevance	Mede se a resposta gerada responde efetivamente à pergunta do utilizador.
Faithfulness	Mede se a resposta gerada é sustentada pela informação presente no contexto fornecido ao modelo.

Tabela 1: Recall@k, Precision@k, HitRate@k e MRR@k são benchmarks de **Retrieval Eval**, enquanto Answer Relevance e Faithfulness são benchmarks de **Generation Eval**.

Definida a metodologia de avaliação, passamos à análise dos resultados obtidos:

Benchmark	Sparse Retrieval	Dense Retrieval	Hybrid Retrieval
Recall@15	61%	**74%**	70%
Precision@15	7%	**8%**	**8%**
HitRate@15	74%	**85%**	81%
MRR@15	52%	56%	**60%**

Tabela 2: Benchmarks avaliados no sistema How To RAG. Primeira fase de avaliação das três estratégias de recuperação.

Benchmark	Sparse Retrieval + Reranker	Dense Retrieval + Reranker	Hybrid Retrieval + Reranker
Recall@15	57%	63%	**79%**
Precision@15	17%	19%	**21%**
HitRate@15	74%	79%	**89%**
MRR@15	67%	71%	**78%**

Tabela 3: Benchmarks avaliados no sistema How To RAG. Segunda fase de avaliação das três estratégias de recuperação.

Benchmark	Qwen2.5 0.5B	Phi-3.5 4B Q4
Answer Relevance	63%	80%
Faithfulness	58%	82%

Tabela 4: Benchmarks avaliados no sistema How To RAG. Avaliação da geração de resposta dos dois modelos. O sistema Retrieval é Hybrid Retrieval + Reranker.

4 Assobio

Passamos agora ao segundo sistema desenvolvido durante o estágio curricular: o sistema Assobio.

4.1 Motivações e Conceitos

Após a conclusão do projeto How To RAG, emergiu, por proposta de um colega, uma nova necessidade: **auditar áudios de chamadas telefónicas realizadas entre postos de trabalho**. Sempre que surgia um problema e era necessário apurar o que tinha acontecido, um operador tinha de ouvir, por vezes, várias gravações até conseguir reunir informação suficiente para compreender a situação e chegar a uma conclusão. Este processo, para além de ser moroso, dependia da análise manual de grandes volumes de áudio.
A solução para esta questão passava por desenvolver um sistema capaz de analisar automaticamente estas gravações e de apoiar o processo de auditoria, facilitando a identificação de informação relevante para a investigação de ocorrências.

Antes de avançarmos para a arquitetura do Assobio, foi necessário compreender como um sinal de áudio é representado digitalmente e de que forma pode ser transformado em informação útil para o seu processamento. **Num sistema computacional, um áudio representa meramente um vetor de valores numéricos** onde cada valor representa a amplitude do sinal num instante específico de amostragem, formando aquilo a que se denomina um *Time-Domain Signal*.

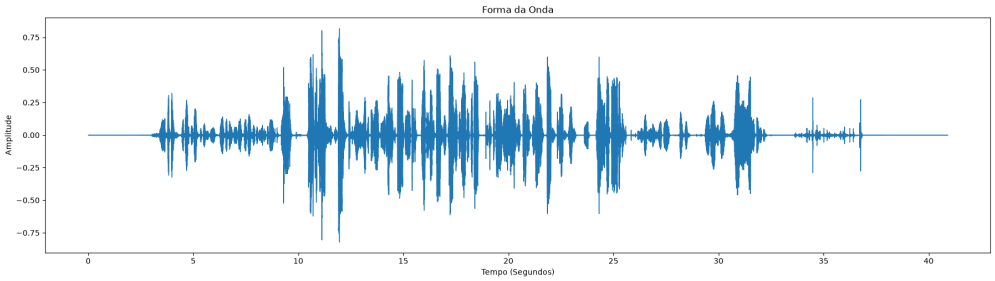


Figura 7: Time-Domain Signal

Esta representação permite reconstruir e reproduzir o som, mas não mostra diretamente como o som é feito. A fala não é apenas uma sequência de variações de amplitude. Também contém padrões de frequência que mudam com o tempo e que são importantes para identificar os sons produzidos. Recorre-se frequentemente à **Transformação de Fourier de Tempo Curto *(Short-Time Fourier Transform, STFT)***, que analisa o sinal através de janelas temporais sucessivas.

$$X(m,\omega) = \sum_{n=-\infty}^{+\infty} x[n]\, w[n - mH]\, e^{-j\omega n}$$

Onde:

- $x[n]$ representa o sinal original do domínio de tempo.
- $w[n - mH]$ representa janela aplicada ao sinal, centrada na posição temporal mH.
- m representa o indíce da janela temporal.
- H representa o *hop length*, ou número de amostras entre janelas consecutivas.
- w representa a frequência angular.
- $e^{-j\omega n}$ representa a componente da Transformação de Fourier que decompõe o sinal em frequências.

O resultado pode ser representado através de um **espectrograma**, no qual o eixo horizontal corresponde ao tempo, o eixo vertical à frequência e a intensidade da cor representa a magnitude ou potência associada a cada componente. Esta representação possibilita a observação da variação da distribuição de energia pelas frequências ao longo da gravação, tornando visíveis padrões não evidentes na forma de onda original.

Ao aprofundar a análise destas representações, foi também possível identificar o **Mel-Espectrograma**, uma técnica utilizada no processamento de áudio e em diversas arquiteturas de *Automatic Speech Recognition (ASR)*. Este resultado é obtido através da aplicação de filtros distribuídos segundo a escala Mel ao espectrograma, agrupando a informação espectral de acordo com uma escala inspirada na forma como o sistema auditivo humano distingue frequências. A representação atribui uma resolução superior às frequências mais baixas e agrupa de forma progressiva as frequências mais elevadas, permitindo obter uma descrição espectral mais compacta e adequada a determinadas tarefas de processamento de áudio.
As frequências para os seres humanos não são percetíveis de maneira linear. A diferença entre 500Hz e 1000Hz pode ser muito mais explícita do que a diferença entre 5000Hz e 5500Hz [11]. **Desta ideia surge o Mel-Espectrograma**.

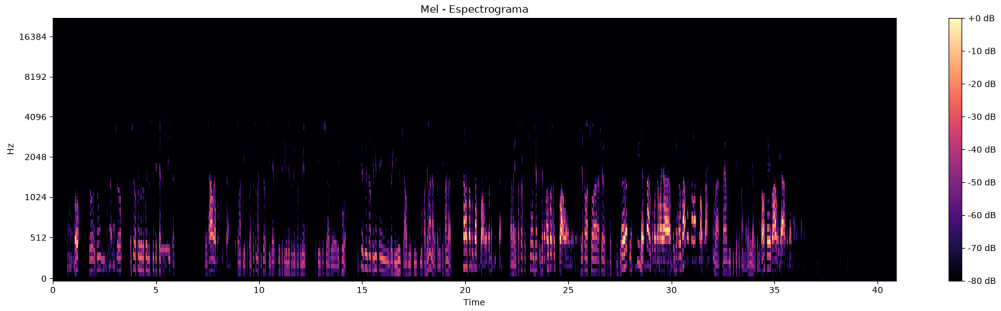


Figura 8: Mel-Espectrograma.

Automatic Speech Recognition (ASR), refere-se à área da investigação responsável pelo desenvolvimento de sistemas com a capacidade de converter linguagem falada em texto. Esta tecnologia representa uma componente fundamental na interação entre humanos e computadores, estando presente em aplicações tão distintas como assistentes de voz, sistemas de comando por voz e serviços de transcrição automática [12].

Desde o início do desenvolvimento do Assobio, tornou-se evidente que esta tecnologia seria indispensável. Para que um modelo de linguagem pudesse apoiar uma auditoria, seria necessário converter primeiro o sinal de áudio numa representação textual. ASR permitiria, assim, transformar as gravações numa forma de informação que pudesse ser posteriormente processada e analisada pelo sistema.
Este foi o nosso primeiro choque de que Inteligência Artificial vai muito para lá da linguagem.

Uma das arquiteturas mais famosas em ASR é a *Encoder-Decoder*. Nesta abordagem, o **Encoder** processa a representação do áudio e transforma-a numa sequência de representações numéricas, denominadas *embeddings* (nada de novo). Estas codificam características relevantes do sinal de entrada e fornecem ao modelo uma representação sobre a qual pode operar. O **Decoder** utiliza essa informação para produzir uma sequência de tokens correspondente à transcrição.

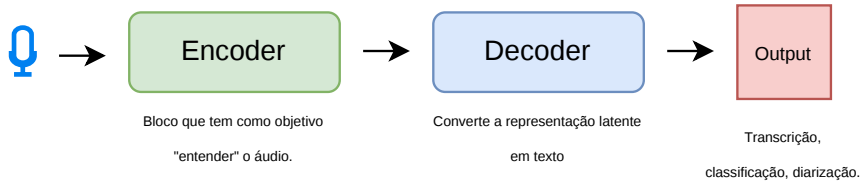


Figura 9: Arquitetura Encoder-Decoder de um modelo ASR.

4.2 Arquitetura

Seguidamente, apresentamos as duas arquiteturas das duas versões do sistema Assobio.

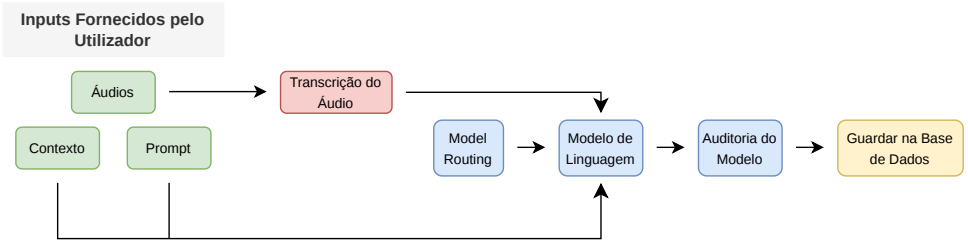


Figura 10: Arquitetura Assobio V1

De acordo com a V1 do Assobio, o processo de auditoria é iniciado com a submissão de um ou mais ficheiros de áudio pelo utilizador. Este define também o contexto e o prompt que orientam a análise. O contexto estabelece os critérios da auditoria, enquanto o prompt especifica as instruções que o modelo deve seguir. Esta separação permite adaptar o sistema a diferentes cenários de utilização, sem limitar o processo de auditoria a um conjunto rígido de regras predefinidas. Em vez de obrigar o utilizador a adaptar-se ao sistema, procurámos atribuir-lhe controlo sobre aquilo que pretende auditar e sobre os critérios que devem orientar essa análise.

Após a submissão dos dados de entrada, cada áudio é encaminhado para o modelo ASR, que se encarrega da conversão do áudio em transcrição textual. De seguida, um mecanismo de **Model Routing** é acionado, selecionando o modelo de linguagem mais adequado entre quatro alternativas disponíveis. Esta seleção é acompanhada por uma decisão relativa à infraestrutura de execução, que encaminha o modelo selecionado para CPU ou GPU em função da sua dimensão e dos recursos computacionais necessários. Em vez de depender de um único modelo e de uma configuração fixa, o Assobio adapta a execução às características dos diferentes modelos e ao hardware disponível, procurando equilibrar a capacidade de análise e o custo computacional.

Terminada a transcrição, a mesma é meticulosamente combinada com o contexto e o prompt definidos previamente pelo utilizador. Estes elementos são fornecidos ao modelo de linguagem, que realiza a auditoria propriamente dita, analisando o conteúdo da conversa à luz dos critérios estabelecidos. Os dados e os resultados do processamento são persistidos numa base de dados SQL, permitindo conservar a informação gerada pelo sistema e disponibilizá-la para consulta posterior.

A segunda versão do Assobio surgiu após a apresentação da V1 ao colega que propôs o projeto, num momento em que a troca de ideias sobre a solução desenvolvida e os objetivos pretendidos se revelou fundamental. Para além deste diálogo, foram disponibilizadas três amostras reais de áudios, permitindo confrontar a abordagem inicial com exemplos concretos e identificar oportunidades de melhoria com base nas necessidades efetivas do processo de auditoria.

Em resposta ao feedback recebido e após uma análise rigorosa das gravações, avançámos para a versão V2, introduzindo melhorias significativas na funcionalidade de auditoria e desenvolvendo uma nova aba, denominada **Chat**.

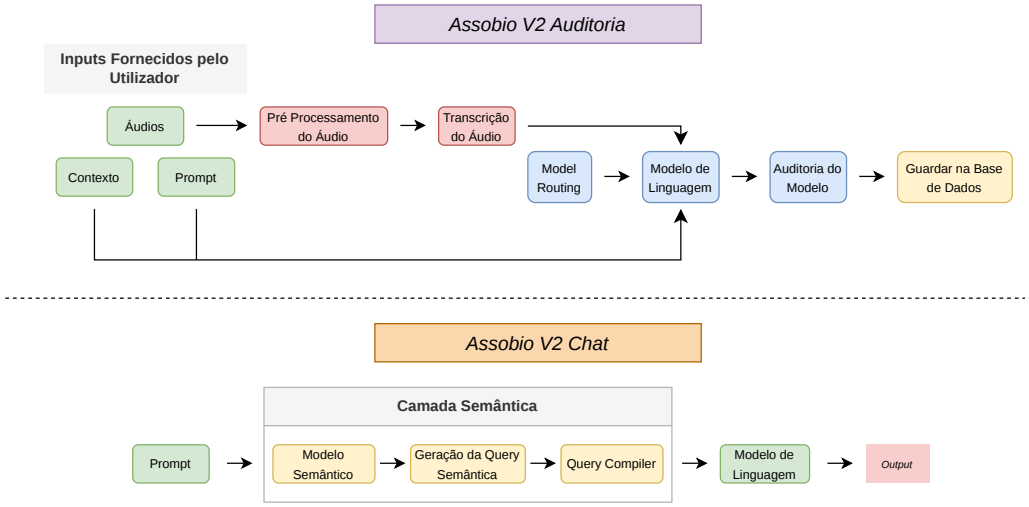


Figura 11: Arquitetura Assobio V2

Introduzimos uma camada de **Pré-Processamento** de áudio na fase de Auditoria, com o objetivo de tornar o sistema mais robusto perante gravações com diferentes características e níveis de qualidade. Para esse efeito, testámos e avaliámos várias abordagens, nomeadamente *Voice Activity Detection (VAD)*, a separação de vozes e sons com *Demucs*, a redução de ruído e a normalização do pico. Esta fase permitiu-nos explorar diferentes formas de preparar o sinal antes da transcrição e assim, em vez de partir do princípio que o áudio recebido está sempre em condições ideais, começámos a preparar o sistema para lidar com a variabilidade encontrada nas gravações reais. O mecanismo de **Model Routing** foi igualmente ampliado, disponibilizando agora um conjunto de 13 modelos de linguagem.

A segunda grande evolução surgiu da necessidade discutida com o colega que propôs o projeto durante a apresentação da V1: **a possibilidade de conversar com um modelo de linguagem sobre o conteúdo das chamadas transcritas**. A ideia era permitir que os utilizadores fizessem perguntas sobre uma ou várias gravações e obtivessem respostas fundamentadas na informação correspondente. Muito semelhante a um problema de contexto como abordámos no primeiro sistema How To RAG. Antes de desenvolver esta funcionalidade, começámos por reforçar a base de dados SQL, enriquecendo a informação nela armazenada, de modo a suportar novas possibilidades de consulta. No entanto, disponibilizar estes dados a um modelo de linguagem suscitava um problema importante:

P4: Como permitir que um modelo responda a perguntas sem lhe conceder liberdade irrestrita para consultar e manusear a base de dados ?

Uma possibilidade seria permitir que o modelo gerasse consultas SQL diretamente a partir das perguntas do utilizador. No entanto, esta estratégia acarreta riscos em termos de fiabilidade e controlo. Por exemplo, o modelo pode gerar consultas sintaticamente inválidas, interpretar incorretamente a estrutura da base de dados ou formular uma consulta que não corresponda à intenção do utilizador. Num sistema destinado a apoiar processos de auditoria, estes erros podem resultar em respostas incompletas ou incorretas, comprometendo a confiança nos resultados.
Foi neste contexto que explorámos a utilização de uma **Camada Semântica** concebida para servir de ponte entre a base de dados e o modelo de linguagem.

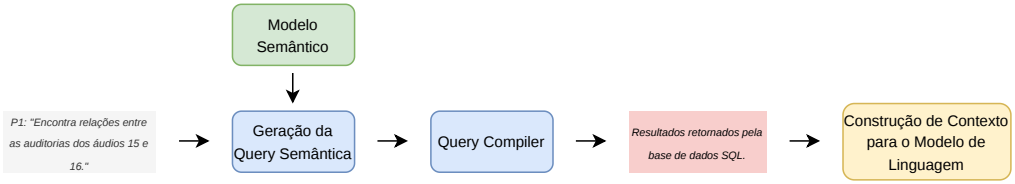


Figura 12: Camada Semântica - O processo começa com a pergunta do utilizador, que é encaminhada para a fase da **Geração da Query Semântica**. Nesta fase, um modelo de linguagem, sujeito a rigorosas restrições de geração, interpreta a intenção da questão e produz uma consulta semântica. Para orientar esta geração, o modelo recebe o **Modelo Semântico**, que estabelece um contrato explícito entre o modelo de linguagem e a base de dados. Este define as operações disponíveis, as colunas acessíveis e o significado de cada campo, delimitando a informação que pode ser consultada e a forma como esta pode ser apresentada. A query semântica produzida é, de seguida, encaminhada para o **Query Compiler**, um compilador altamente restritivo que valida a query e a traduz para uma query SQL. Só após esta tradução é que a query SQL é executada na base de dados. Os resultados obtidos são posteriormente incorporados no contexto do modelo de linguagem utilizado na aba *Chat*.

Desta maneira, em vez de exigir que o modelo conheça diretamente a estrutura SQL e construa consultas de forma livre, a Camada Semântica aplica um mecanismo seguro e controlado que permite fornecer um válido e conciso contexto ao modelo de linguagem. A introdução desta camada deu origem à nova funcionalidade *Chat*, que permite explorar os dados do Assobio através de perguntas em linguagem natural.

Concluindo, a V2 não se limita a melhorar o processamento das gravações, mas sim a transformar a informação recolhida pelo sistema numa fonte de conhecimento que o utilizador pode consultar de forma mais flexível, complementando as auditorias estruturadas com uma experiência de chatbot.

4.3 Avaliação

Após a construção do sistema, iniciámos a fase de avaliação, com o objetivo de identificar os fatores que mais impactam o seu desempenho. Considerámos prioritário avaliar, desde logo, a transcrição do modelo ASR, visto que a mesma constitui a base de todo o processo de auditoria. Para o Assobio, o objetivo não era obter uma transcrição perfeita, mas sim preservar o significado e a informação relevante contida nas gravações, tanto quanto possível. Este ponto é importante, visto que nem todos os erros de transcrição têm o mesmo impacto numa auditoria e uma alteração aparentemente pequena pode ser suficiente para alterar o sentido de uma frase ou comprometer a interpretação de um acontecimento.

À semelhança do processo de avaliação do How To RAG, recorremos à construção de um *Golden Dataset* e à definição de *Benchmarks* para comparar diferentes configurações. Neste caso, construímos manualmente um conjunto de dados composto por gravações de chamadas telefónicas obtidas no YouTube, pelas respetivas transcrições de referência e pela duração de cada áudio.

Aproveitámos também esta avaliação para comparar dois modelos ASR: Whisper Large [13] e o WhisperLv3-PT [14]. Este último modelo foi desenvolvido no âmbito do projeto CAMÕES [15], tendo sido treinado com dados de áudio abrangentes, que incluem variedades do Português Europeu, Brasileiro, Africano e Asiático. Os benchmarks definidos para esta avaliação encontram-se apresentados na tabela seguinte:

Benchmark	Objetivo
World Error Rate (WER)	Mede a proporção de erros ao nível das palavras.
Character Error Rate (CER)	Mede a proporção de erros ao nível dos caracteres.
Real Time Factor (RTF)	Mede o tempo de processamento relativamente à duração do áudio.
Latência	Mede o tempo entre o input e a resposta do sistema.

Tabela 5: Benchmarks avaliados no sistema Assobio.

No momento da avaliação da V1, a camada de Pré-processamento de Audio ainda não estava integrada no sistema. Por conseguinte, a nossa análise ficou restrita à comparação de duas estratégias de Decoding dos modelos ASR: **Greedy Decoding** e **Beam Search Decoding**.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.39**	0.54
CER	**0.19**	0.44
RTF	0.34	**0.18**
Latência	14.71	**7.76**

Tabela 6: Resultados dos benchmarks com a estratégia **Greedy Decoding**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.36**	0.49
CER	**0.18**	0.38
RTF	0.38	**0.31**
Latência	16.33	**12.72**

Tabela 7: Resultados dos benchmarks com a estratégia **Beam Search Decoding = 5**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.35**	0.48
CER	**0.18**	0.36
RTF	0.49	**0.40**
Latência	21.23	**16.48**

Tabela 8: Resultados dos benchmarks com a estratégia **Beam Search Decoding = 10**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Terminámos aqui a avaliação da V1 do Assobio concluindo que o modelo **Whisper Large obtinha melhor performance na transcrição** e o modelo **WhisperLv3-PT obtinha melhores resultados no tempo de processamento**.

À medida que o Assobio evoluiu para a versão 2, surgiram novas funcionalidades e, com elas, novas necessidades de avaliação. Foram identificados dois pontos críticos no sistema: a transcrição do áudio para texto, cuja avaliação já tinha sido iniciada na primeira versão e que poderia ser aprofundada com a introdução de diferentes técnicas de pré-processamento; e a geração de consultas semânticas, responsável por estabelecer a ligação entre as perguntas do utilizador e a informação armazenada na base de dados.

O primeiro ponto corresponde à transcrição do áudio. Na V1, a avaliação tinha-se centrado na comparação de estratégias de decoding dos modelos ASR. Com a introdução da camada de pré-processamento na V2, tornou-se possível estudar uma nova variável: o impacto das transformações aplicadas ao áudio antes da transcrição.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.46**	0.61
CER	**0.23**	0.51
RTF	0.48	**0.41**
Latência	20.72	**16.73**

Tabela 9: Resultados dos benchmarks com **Pré Processamento (Voice Activity Detection) + Beam Search Decoding = 5**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.41**	0.59
CER	**0.22**	0.45
RTF	0.44	**0.33**
Latência	19.19	**13.87**

Tabela 10: Resultados dos benchmarks com **Pré Processamento (Demucs) + Beam Search Decoding = 5**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Benchmark	Whisper Large	WhisperLv3-PT
WER	0.52	**0.49**
CER	**0.30**	0.38
RTF	0.48	**0.25**
Latência	21.49	**10.92**

Tabela 11: Resultados dos benchmarks com **Pré Processamento (Noise Reducer) + Beam Search Decoding = 5**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.37**	0.51
CER	**0.18**	0.40
RTF	0.46	**0.30**
Latência	19.26	**12.49**

Tabela 12: Resultados dos benchmarks com **Pré Processamento (Peak Normalization) + Beam Search Decoding = 5**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.38**	0.51
CER	**0.21**	0.39
RTF	0.38	**0.28**
Latência	16.42	**12.20**

Tabela 13: Resultados dos benchmarks com **Pré Processamento (Demucs + Peak Normalization + Aumento Controlado do Volume) + Beam Search Decoding = 5**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Benchmark	Whisper Large	WhisperLv3-PT
WER	**0.46**	0.43
CER	**0.24**	0.35
RTF	0.35	**0.24**
Latência	15.39	**10.43**

Tabela 14: Resultados dos benchmarks com **Pré Processamento (Voice Ativity Detection + Demucs + Peak Normalization + Aumento Controlado do Volume) + Beam Search Decoding = 5**.
Menor é melhor. Latência apresentada em segundos. Todos os valores correspondem à média.

Concluímos que uma melhoria percetível para o ouvido humano não se traduz, necessariamente, numa melhoria para o modelo ASR. Determinadas técnicas de pré-processamento podem tornar o áudio aparentemente mais limpo ou perceptível, mas sem melhorar a qualidade da transcrição, podendo mesmo prejudicá-la. **O que soa melhor ao ouvido humano não é, necessariamente, o que permite ao modelo reconhecer melhor a fala**.

O segundo ponto crítico surgiu com a introdução da Camada Semântica na secção *Chat*. Como explicado anteriormente, esta camada utiliza um modelo de linguagem para transformar a pergunta do utilizador numa consulta semântica, que é posteriormente interpretada pelo compilador de consultas e traduzida para SQL. Esta fase é determinante: se o modelo interpretar incorretamente a questão ou gerar uma estrutura semântica inválida, todo o processo de consulta pode falhar, mesmo que a base de dados contenha a informação desejada. Ao contrário de uma resposta textual, que pode ter diferentes graus de adequação, uma consulta destinada a ser interpretada por um compilador deve respeitar um conjunto explícito de operações, campos e regras. A fluência linguística do modelo é irrelevante por si só se o resultado não cumprir esse contrato.

Neste contexto, nestas semanas de avaliação, surgiu uma nova abordagem aos modelos de linguagem, designada **System One Models**, apresentada pela TypeSafeAI através do modelo **Jev** [16]. Esta linha de desenvolvimento privilegia respostas rápidas e decisões fiáveis em tarefas específicas, sendo mais adequada a cenários em que a consistência dos resultados é mais importante do que a capacidade de gerar respostas abertas e elaboradas.
Para o projeto Assobio, esta proposta era particularmente relevante, visto que a geração de consultas semânticas consiste numa tarefa estruturada com um espaço relativamente restrito de resultados possíveis, sendo essencial a capacidade de seguir regras e produzir resultados válidos. Embora o Jev fosse um modelo fechado, a posterior emergência de alternativas de open source, como o **Laya**, possibilitou a exploração desta abordagem sem depender exclusivamente de uma solução proprietária. Por conseguinte, decidimos avaliar se estes modelos apresentavam vantagens na tarefa específica de geração de consultas semânticas.

Para realizar esta comparação, foi construído um Synthetic Golden Dataset, seguindo a estrutura ilustrada na figura 5. Recorremos à geração sintética de casos de teste que reproduzem diferentes formas de interação com a camada semântica. Cada exemplo inclui o prompt introduzido pelo utilizador, a respetiva versão em inglês e o resultado semântico esperado, o qual deverá ser compatível com as operações suportadas pelo Query Compiler. Com este conjunto de dados, avaliámos três modelos: o Mistral 7B Q4 [17], o Laya Multilingue e o Laya Inglês [18].

Modelo	Accuracy	Latência
Mistral 7B Q4	**0.857**	2.726
Laya Multilingue	0.507	**0.04**
Laya Inglês	0.650	**0.04**

Tabela 15: Large Language Model vs System One Model
Aplicado à fase da Geração da Query Semântica num sistema com Camada Semântica.

Embora o System One Model tenha apresentado resultados inferiores na vertente *Accuracy*, acreditamos que esta abordagem tem o potencial de se tornar uma mais-valia no desenvolvimento de sistemas de Inteligência Artificial. À medida que forem surgindo modelos mais robustos, capazes de garantir uma maior precisão, a sua rapidez e orientação para tarefas específicas poderão torná-los particularmente interessantes em cenários que exijam decisões rápidas, saídas estruturadas e uma elevada fiabilidade. Os resultados obtidos não justificam, por ora, a sua adoção em vez das alternativas avaliadas, mas reforçam o interesse em acompanhar a evolução desta abordagem e explorar a sua aplicação futura em sistemas.

5 Conclusão

Concluímos, assim, a apresentação do trabalho desenvolvido ao longo de quatro meses de estágio. Mais do que explorar tecnologias emergentes, procurámos demonstrar que a adoção de Inteligência Artificial nas organizações não deve ficar pelo acesso às ferramentas existentes, mas evoluir para a capacidade de construir soluções próprias, orientadas para problemas concretos. Este trabalho representa um passo nessa direção.

O futuro já começou. Estamos apenas no início.

Referências

[1] Comissão Europeia. *State of the Digital Decade 2026 - Closing structural gaps and mobilising investments for 2030 and beyond*. URL: https://digital-strategy.ec.europa.eu/en/library/state-digital-decade-2026-closing-structural-gaps-and-mobilising-investments-2030-and-beyond (acedido em 07/10/2026).

[2] David Roodman Luke Emberson. *The plunging price of thought*. URL: https://epoch.ai/publications/the-plunging-price-of-thought (acedido em 07/10/2026).

[3] António Simões Eliezer Carvalho. *A Three-Dimensional Analysis for LLM Deployment*. URL: https://github.com/Eliezer-Carvalho/Curricular-Internship-Infraestruturas-Portugal/blob/master/A%20Three-Dimensional%20Analysis%20for%20LLM%20Deployment/A%20Three-Dimensional%20Analysis%20for%20LLM%20Deployment.pdf (acedido em 08/10/2026).

[4] António Simões Eliezer Carvalho. *Context Engineering*. URL: https://github.com/Eliezer-Carvalho/Curricular-Internship-Infraestruturas-Portugal/blob/master/Context%20Engineering/Context%20Engineering.pdf (acedido em 08/10/2026).

[5] António Simões Eliezer Carvalho. *Synthetic Data*. URL: https://github.com/Eliezer-Carvalho/Curricular-Internship-Infraestruturas-Portugal/blob/master/Synthetic%20Data/Synthetic%20Data.pdf (acedido em 08/10/2026).

[6] António Simões Eliezer Carvalho. *How To RAG*. URL: https://github.com/Eliezer-Carvalho/Curricular-Internship-Infraestruturas-Portugal/blob/master/How%20To%20RAG/How%20To%20RAG.pdf (acedido em 09/10/2026).

[7] Microsoft. *microsoft/phi-4*. URL: https://huggingface.co/microsoft/phi-4 (acedido em 09/10/2026).

[8] Mistral. *mistralai/Mistral-7B-Instruct-v0.3*. URL: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3 (acedido em 09/10/2026).

[9] Alibaba Cloud. *Qwen/Qwen2.5-0.5B*. URL: https://huggingface.co/Qwen/Qwen2.5-0.5B (acedido em 09/10/2026).

[10] Microsoft. *microsoft/Phi-3.5-mini-instruct*. URL: https://huggingface.co/microsoft/Phi-3.5-mini-instruct (acedido em 09/10/2026).

[11] António Simões Eliezer Carvalho. *Automatic Speech Recognition*. URL: https://github.com/Eliezer-Carvalho/Curricular-Internship-Infraestruturas-Portugal/blob/master/Automatic%20Speech%20Recognition/Automatic%20Speech%20Recognition.pdf (acedido em 10/10/2026).

[12] António Simões Eliezer Carvalho. *Automatic Speech Recognition*. URL: https://github.com/Eliezer-Carvalho/Curricular-Internship-Infraestruturas-Portugal/blob/master/Automatic%20Speech%20Recognition/Automatic%20Speech%20Recognition.pdf (acedido em 10/10/2026).

[13] openai. *openai/whisper-large-v3*. URL: https://huggingface.co/openai/whisper-large-v3 (acedido em 10/10/2026).

[14] inesc-id. *inesc-id/WhisperLv3-X-PT-All*. URL: https://huggingface.co/inesc-id/WhisperLv3-X-PT-All (acedido em 10/10/2026).

[15] Catarina Botelho Anna Pompili Rubén Solera-Ureña Sérgio Paulo Mariana Julião Thomas Rolland John Mendonça Diogo Pereira Isabel Trancoso Alberto Abad Carlos Carvalho Francisco Teixeira. "CAMÕES: A Comprehensive Automatic Speech Recognition Benchmark for European Portuguese". Em: (2025).

[16] TypeSafe AI. *Introducing System One Models Jev*. URL: https://typesafe.ai/blog/introducing-system-one-models-and-jev (acedido em 11/10/2026).

[17] Mistral. *mistralai/Mistral-7B-Instruct-v0.3*. URL: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3 (acedido em 09/10/2026).

[18] convaiinnovations. *convaiinnovations/laya*. URL: https://huggingface.co/convaiinnovations/laya (acedido em 09/10/2026).